

UNIVERSIDADE DO ESTADO DA BAHIA – UNEB

Departamento de Ciências Exatas e da Terra – Campus I

Tópicos Especiais em Engenharia de Software

Prof. Eduardo Manuel de Freitas Jorge

SPARK

Sistema de Busca de Informações de Pesquisadores
Baseado em Full-Text Search e LLM

Sprint I – Especificação do Projeto

SPK-71: Descrição do Sistema e Especificação de Requisitos
(Histórias do Usuário com Critérios de Aceitação)

Equipe: Calvin Albuquerque
Glenda Santana

Versão: 1.0

Data: 18 de maio de 2026

Salvador, Bahia – 2026

Sumário

1	Introdução	1
2	Descrição do Sistema	1
2.1	Contexto e Motivação	1
2.2	Propósito	1
2.3	Escopo	2
2.4	Sistemas Correlatos	2
2.5	Tecnologias Utilizadas	3
3	Especificação de Requisitos	4
3.1	Metodologia	4
3.2	Atores do Sistema	4
3.3	Histórias do Usuário	5
	US-01 – Busca Textual por Palavras-Chave	5
	US-02 – Busca Semântica por Linguagem Natural	6
	US-03 – Orquestração do ETL dos Dados Lattes	7
	US-04 – Geração de Dados Analíticos para Power BI	8
4	Casos de Uso	9
4.1	Diagrama de Casos de Uso	9
4.2	Atores	9
4.3	Descrição dos Casos de Uso	10
	UC-01 – Buscar Produções	10
	UC-02 – Filtrar Resultados	11
	UC-03 – Visualizar Detalhes da Produção	12
	UC-04 – Visualizar Perfil do Pesquisador	12
	UC-05 – Gerenciar Pesquisadores	13
	UC-06 – Exportar Dados para Power BI	14
5	Projeto Arquitetural	15
5.1	Camada Operacional	15
5.2	Camada Back-End	16

5.3	Camada Front-End	16
5.4	Protocolos de Comunicação	17
6	Esquema do Banco de Dados	18
6.1	Diagrama Entidade-Relacionamento	18
6.2	Descrição das Tabelas	19
6.3	Decisões Técnicas do Modelo	19
7	Rastreabilidade dos Requisitos	20
8	Considerações Finais	20

1 Introdução

Este documento apresenta a especificação do **SPARK**, um sistema de busca avançada de produções científicas de pesquisadores desenvolvido como Projeto Integrador da disciplina de Tópicos Especiais em Engenharia de Software da Universidade do Estado da Bahia (UNEB).

O documento está organizado em duas seções principais: a **Descrição do Sistema**, que contextualiza o problema, os objetivos e a solução proposta; e a **Especificação de Requisitos**, que apresenta as funcionalidades do sistema na forma de Histórias do Usuário com seus respectivos Critérios de Aceitação.

2 Descrição do Sistema

2.1 Contexto e Motivação

As universidades brasileiras produzem um volume significativo de pesquisa científica, registrada nos currículos Lattes de seus pesquisadores. No entanto, a ausência de ferramentas de busca semântica e contextual dificulta o mapeamento de competências, a identificação de colaboradores e a análise de tendências de pesquisa institucionais.

O sistema SPARK surge para atender a essa necessidade no contexto da UNEB, oferecendo uma plataforma que centraliza e torna navegável a produção científica dos pesquisadores de forma inteligente.

2.2 Propósito

O SPARK tem como propósito principal orquestrar o processo de **ETL** (Extração, Transformação e Carga) dos dados dos currículos Lattes – disponíveis em formato XML – para uma base de dados relacional, enriquecendo as informações com dados bibliométricos externos (Qualis CAPES, Fator de Impacto JCR e DOI).

Sobre essa base estruturada, o sistema disponibiliza duas modalidades de busca:

- **Busca textual (Full-Text Search):** baseada em indexação nativa do PostgreSQL via `tsvector`, permitindo buscas por palavras-chave com suporte a operadores bo-

oleanos.

- **Busca semântica:** baseada em vetores matemáticos (*embeddings*) gerados por Modelos de Linguagem (LLM), armazenados via extensão **pgvector** no Supabase, possibilitando encontrar produções por contexto e significado.

2.3 Escopo

O sistema SPARK abrange os seguintes módulos:

1. **Camada Operacional (ETL):** pipeline Apache Hop para leitura, transformação e carga dos XMLs do Lattes no banco de dados relacional, incluindo integração com bases externas (Qualis CAPES, JCR, DOI).
2. **Camada de Back-End:** API RESTful desenvolvida em Python com FastAPI, hospedada no Railway, responsável pelas regras de negócio, geração de *embeddings* (via Sentence-Transformers com *fallback* para OpenAI/Langchain) e exportação de dados analíticos em CSV.
3. **Camada de Banco de Dados:** Supabase (PostgreSQL gerenciado) com a extensão **pgvector** habilitada, responsável pelo armazenamento relacional e vetorial.
4. **Camada de Front-End:** aplicação web desenvolvida em Next.js, hospedada no Vercel, que consome a API REST para exibir os resultados de busca em interface responsiva com indicadores Qualis e JCR.
5. **Camada Analítica:** arquivos CSV gerados pelo back-end e consumidos pelo Power BI para elaboração de painéis e relatórios analíticos.

2.4 Sistemas Correlatos

Para embasar o projeto, foram analisados sistemas com objetivos similares:

Tabela 1: Sistemas correlatos ao SPARK

Sistema	Descrição
SIMCC (UESC)	Sistema de Mapeamento de Competências da Bahia, disponível em simcc.uesc.br .
SOMOS (UFMG)	Sistema de mapeamento de competências da UFMG para incrementar a interação com instituições públicas e privadas, disponível em somos.ufmg.br .

2.5 Tecnologias Utilizadas

Tabela 2: Tecnologias e ferramentas do projeto SPARK

Camada	Tecnologia	Finalidade
Orquestração	Apache Hop	Pipeline ETL dos XMLs do Lattes
Banco de Dados	Supabase	PostgreSQL gerenciado com pg-vector
Back-End	Python / FastAPI	API RESTful e lógica de negócios
Back-End	Railway	Hospedagem gratuita do back-end
IA / LLM	Sentence-Transformers / OpenAI	Geração de embeddings
Front-End	Next.js	Interface de busca e visualização
Front-End	Vercel	Hospedagem gratuita do front-end
Analítico	Power BI	Dashboards e relatórios
Versionamento	Git / GitHub	Controle de versão do código
Containerização	Docker	Ambiente de desenvolvimento local

3 Especificação de Requisitos

3.1 Metodologia

Os requisitos foram levantados e documentados no formato de **Histórias do Usuário** (*User Stories*), conforme a metodologia ágil Scrum adotada no projeto. Cada história segue a estrutura:

“Como [ator], quero [funcionalidade], para [objetivo/valor].”

Cada história possui **Critérios de Aceitação** mensuráveis que definem quando a funcionalidade está concluída, e uma **Definição de Pronto** (DoD) que estabelece os requisitos de qualidade e entrega.

3.2 Atores do Sistema

- **Usuário:** pesquisador, gestor ou qualquer pessoa que utiliza a interface web para buscar produções científicas.
- **Engenheiro de Dados:** responsável pela orquestração do pipeline de ETL via Apache Hop.
- **Analista de Dados:** responsável pela criação de dashboards no Power BI a partir dos CSVs gerados.

3.3 Histórias do Usuário

US-01 Busca Textual por Palavras-Chave

US-01 – Busca Textual por Palavras-Chave	
Como	Usuário do sistema
Quero	Buscar produções científicas por palavras-chave
Para	Encontrar artigos e produções que contenham os termos pesquisados no título
Critérios de Aceitação	
CA-01.1	O sistema deve retornar resultados que contenham as palavras-chave no título da produção.
CA-01.2	A busca deve suportar operadores básicos de Full-Text Search (AND, OR, NOT) via índice <code>tsvector</code> do PostgreSQL.
CA-01.3	Os resultados devem ser paginados, exibindo no máximo 20 itens por página.
CA-01.4	O tempo de resposta da API deve ser inferior a 30 segundos.
CA-01.5	Cada resultado deve exibir: título, nome do veículo, ano de publicação, estrato Qualis e Fator de Impacto JCR (quando disponíveis).
Definição de Pronto (DoD)	
DoD	Código no Git com commit referenciando esta issue; endpoint <code>POST /api/search/text</code> funcional e documentado no Swagger; testes unitários passando; demonstrado na Sprint Review.

US-02 Busca Semântica por Linguagem Natural

US-02 – Busca Semântica por Linguagem Natural

Como Usuário do sistema

Quero Realizar buscas usando linguagem natural

Para Encontrar produções cujo contexto e significado correspondam à minha pesquisa, sem precisar usar palavras exatas

Critérios de Aceitação

CA-02.1 O sistema deve converter a consulta do usuário em vetor (*embedding*) usando o modelo local Sentence-Transformers.

CA-02.2 O sistema deve calcular a similaridade de cosseno via **pgvector** no Supabase e retornar os resultados mais similares ranqueados no topo.

CA-02.3 Em caso de falha do modelo local, o sistema deve acionar automaticamente o *fallback* para OpenAI/Langchain.

CA-02.4 O score de similaridade deve ser visível na resposta JSON e exibido na interface.

CA-02.5 O tempo de resposta deve ser inferior a 5 segundos para o modelo local.

CA-02.6 Testes integrados devem validar os dois modos (local e *fallback*).

Definição de Pronto (DoD)

DoD Código no Git com commit referenciando esta issue; endpoint POST `/api/search/semantic` funcional; testes integrados passando (local e *fallback*); tempo de resposta validado; demonstrado na Sprint Review.

US-03 Orquestração do ETL dos Dados Lattes**US-03 – Orquestração do ETL dos Dados Lattes**

Como Engenheiro de Dados

Quero Orquestrar a extração e carga dos dados dos currículos Lattes em formato XML via Apache Hop

Para Popular o banco de dados relacional no Supabase com as informações dos pesquisadores e suas produções científicas

Critérios de Aceitação

CA-03.1 O pipeline Hop deve ler diretórios contendo arquivos XML do Lattes.

CA-03.2 As tags de Pesquisador e Produções devem ser mapeadas e inseridas nas respectivas tabelas do banco.

CA-03.3 O pipeline deve realizar UPSERT (inserção ou atualização) para evitar duplicações.

CA-03.4 Após a carga, o pipeline deve integrar os dados com bases externas: Qualis CAPES (por ISSN/nome do veículo), DOI e Fator de Impacto JCR.

CA-03.5 O pipeline deve gerar um log com os registros que não obtiveram correspondência nas bases externas.

CA-03.6 O pipeline deve ser testado com no mínimo 10 currículos reais.

Definição de Pronto (DoD)

DoD Código do pipeline no Git com README de instalação e execução; executado com sucesso com 10 currículos reais; demonstrado na Sprint Review.

US-04 Geração de Dados Analíticos para Power BI

US-04 – Geração de Dados Analíticos para Power BI

Como Analista de Dados

Quero Acessar arquivos CSV gerados pelo back-end com as dimensões e fatos do projeto

Para Criar dashboards interativos no Power BI com análises sobre a produção científica da instituição

Critérios de Aceitação

CA-04.1 O back-end deve exportar as dimensões **Pesquisador** e **Quadrienal-Ano** e o fato relativo à produção científica em arquivos CSV separados.

CA-04.2 Os CSVs devem ter tipagem correta (datas, inteiros e textos) para leitura direta no Power BI sem transformações adicionais.

CA-04.3 O endpoint GET `/api/export/csv` deve retornar o arquivo CSV para download.

CA-04.4 O painel do Power BI deve apresentar gráficos de publicações por ano, por pesquisador e distribuição por estrato Qualis.

CA-04.5 O painel deve permitir filtragem por período e área do conhecimento.

Definição de Pronto (DoD)

DoD Arquivo .pbix no Git; CSVs gerados e validados no Power BI; painel com gráficos e filtros funcionais; demonstrado na Sprint Review.

4 Casos de Uso

4.1 Diagrama de Casos de Uso

O diagrama abaixo representa as interações dos atores com o sistema SPARK do ponto de vista do usuário final.

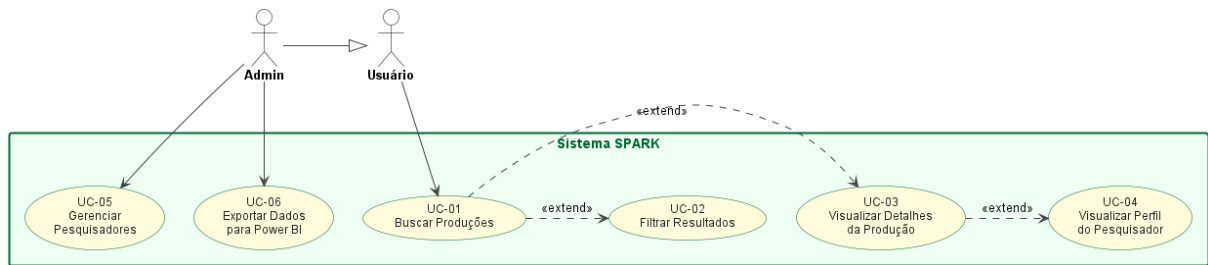


Figura 1: Diagrama de Casos de Uso do sistema SPARK

4.2 Atores

Tabela 3: Atores do sistema SPARK

Ator	Descrição
Usuário	Pesquisador, gestor ou qualquer pessoa que acessa a interface web para realizar buscas e navegar pelas produções científicas. Não requer autenticação para as funcionalidades de busca.
Admin	Usuário com permissões elevadas, responsável por gerenciar os pesquisadores cadastrados no sistema e exportar os dados analíticos. Herda todas as ações do Usuário.

4.3 Descrição dos Casos de Uso

UC-01 Buscar Produções

UC-01 – Buscar Produções	
Ator Principal	Usuário
Pré-condição	O banco de dados deve estar populado com ao menos uma produção científica.
Pós-condição	O usuário visualiza uma lista paginada de produções ordenadas por relevância.
Fluxo Principal	
1	O usuário acessa a interface web e digita um termo ou pergunta na barra de busca.
2	O usuário seleciona o modo de busca: Textual (por palavras-chave) ou Semântico (por linguagem natural).
3	O usuário aciona a busca.
4	O sistema processa a consulta e retorna os resultados em cards, exibindo título, veículo, ano, estrato Qualis e Fator de Impacto JCR de cada produção.
Fluxo de Exceção E1 – Nenhum resultado	
4e	O sistema exibe a mensagem “Nenhuma produção encontrada para a sua busca.” e sugere termos alternativos.
Extensões	
UC-02 (Filtrar Resultados) e UC-03 (Visualizar Detalhes da Produção) podem ser acionados a partir deste caso de uso.	

UC-02 Filtrar Resultados

UC-02 – Filtrar Resultados		«extend» UC-01
Ator Principal	Usuário	
Pré-condição	UC-01 deve ter sido executado e retornado resultados.	
Pós-condição	Os resultados são reapresentados conforme os critérios selecionados.	
Fluxo Principal		
1	O usuário seleciona um ou mais filtros disponíveis: ano de publicação, estrato Qualis, faixa de Fator de Impacto JCR ou área do conhecimento.	
2	O sistema aplica os filtros e atualiza a listagem de resultados sem recarregar a página.	
Fluxo de Exceção E1 – Nenhum resultado após filtro		
2e	O sistema exibe a mensagem “Nenhuma produção encontrada com os filtros selecionados.”	

UC-03 Visualizar Detalhes da Produção

UC-03 – Visualizar Detalhes da Produção		«extend» UC-01
Ator Principal	Usuário	
Pré-condição	A produção deve estar cadastrada no banco.	
Pós-condição	O usuário visualiza todas as informações disponíveis sobre a produção selecionada.	
Fluxo Principal		
1	O usuário clica em um card de produção na lista de resultados.	
2	O sistema exibe: título completo, ano, nome do veículo, DOI (como link externo), estrato Qualis, Fator de Impacto JCR e nome do pesquisador responsável.	
Extensões		
	UC-04 (Visualizar Perfil do Pesquisador) pode ser acionado a partir deste caso de uso.	

UC-04 Visualizar Perfil do Pesquisador

UC-04 – Visualizar Perfil do Pesquisador		«extend» UC-03
Ator Principal	Usuário	
Pré-condição	O pesquisador deve estar cadastrado no banco.	
Pós-condição	O usuário visualiza o perfil completo do pesquisador com suas produções.	
Fluxo Principal		
1	O usuário clica no nome do pesquisador na página de detalhes de uma produção.	
2	O sistema exibe: nome completo, resumo do currículo Lattes, total de produções, distribuição por estrato Qualis e lista paginada de todas as produções do pesquisador.	

UC-05 Gerenciar Pesquisadores**UC-05 – Gerenciar Pesquisadores****Ator Principal** Admin**Pré-condição** O Admin deve estar autenticado no sistema.**Pós-condição** O pesquisador é cadastrado, atualizado ou removido do banco de dados.**Fluxo Principal – Cadastrar**

- 1 O Admin acessa o painel administrativo.
- 2 O Admin preenche o formulário com os dados do pesquisador (nome completo e identificador Lattes).
- 3 O sistema valida e persiste o pesquisador no banco de dados.

Fluxo Alternativo A – Editar

- 2a O Admin seleciona um pesquisador existente, edita os dados e confirma a atualização.

Fluxo Alternativo B – Remover

- 2b O Admin seleciona um pesquisador e confirma a remoção. O sistema remove o pesquisador e todas as suas produções e vetores associados.

UC-06 Exportar Dados para Power BI**UC-06 – Exportar Dados para Power BI****Ator Principal** Admin**Pré-condição** O Admin deve estar autenticado e o banco deve estar populado com produções.**Pós-condição** Arquivos CSV com dimensões e fatos são gerados e disponibilizados para download.**Fluxo Principal**

- 1 O Admin acessa a seção de exportação no painel administrativo.
- 2 O Admin seleciona os dados a exportar: dimensão Pesquisador, dimensão Quadrienal-Ano ou fato Produção Científica.
- 3 O sistema gera o arquivo CSV com tipagem correta e disponibiliza para download.
- 4 O Admin importa o CSV no Power BI para elaborar os dashboards analíticos.

5 Projeto Arquitetural

A arquitetura do sistema SPARK é dividida em três camadas independentes e desacopladas, comunicando-se através de protocolos padrão. `SPK-73_arquitetura.puml`.

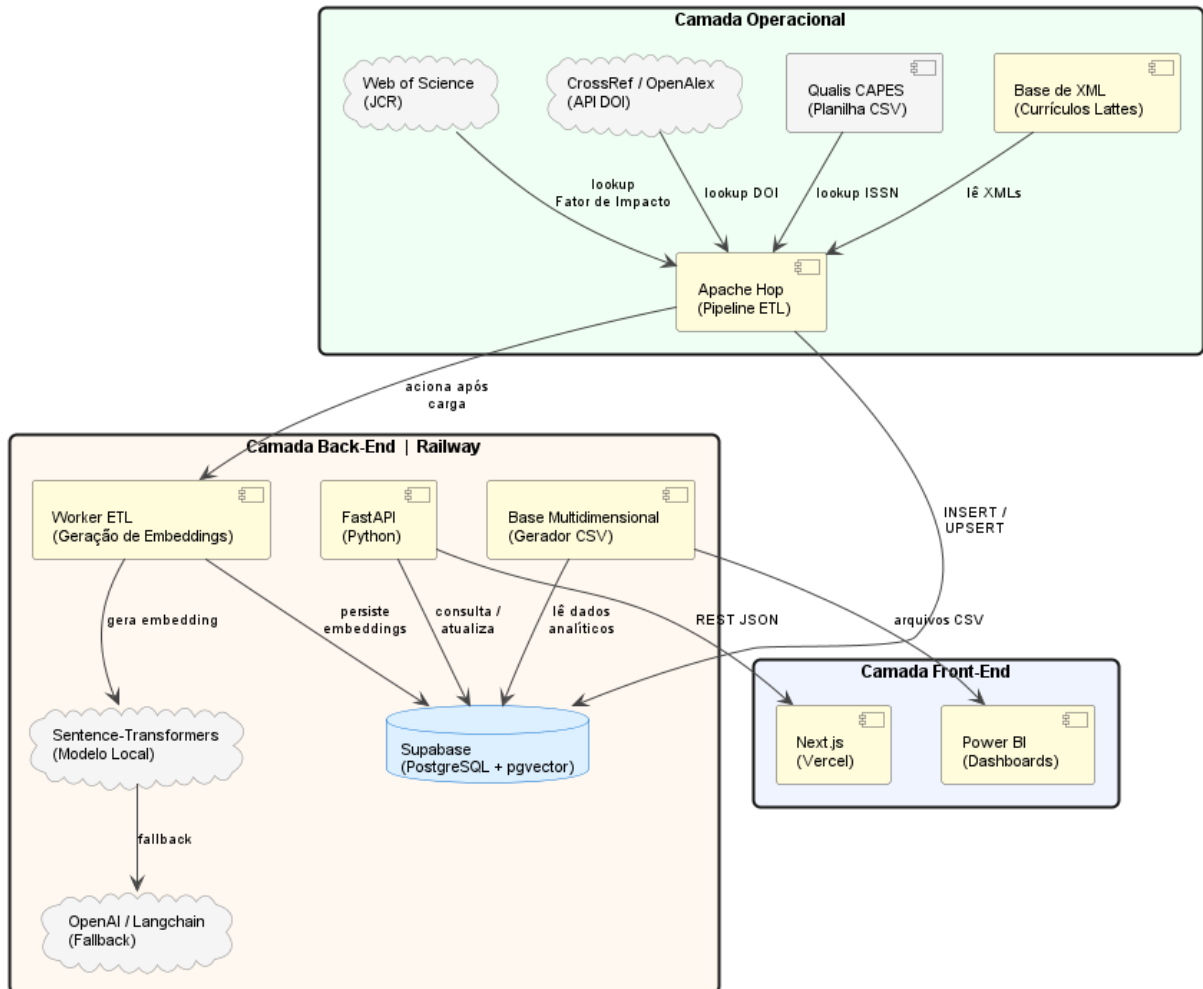


Figura 2: Diagrama Arquitetural do sistema SPARK

5.1 Camada Operacional

Responsável pela extração, transformação e carga dos dados brutos. É composta pelo repositório de arquivos XML dos currículos Lattes e pela ferramenta de orquestração **Apache Hop**. O pipeline ETL executa as seguintes etapas:

1. Leitura dos arquivos XML do Lattes do diretório configurado.
2. Transformação e higienização dos dados (nós XML para colunas relacionais, remoção

de anomalias e padronização de tipos).

3. Enriquecimento com bases externas:

- **Qualis CAPES** (planilha CSV oficial): classificação dos periódicos por estrato (A1, A2, A3, A4, B1, B2...) via ISSN ou nome do veículo.
- **CrossRef / OpenAlex** (API): obtenção do DOI a partir do título da produção.
- **Web of Science / JCR**: Fator de Impacto do periódico via ISSN.

4. Carga via UPSERT nas tabelas `Pesquisadores` e `Producoes` do Supabase.

5. Acionamento automático do Worker de geração de embeddings ao término da carga.

5.2 Camada Back-End

Hospedada no **Railway**, é o núcleo do sistema. Composta por:

- **Supabase (PostgreSQL + pgvector)**: banco de dados gerenciado que armazena tanto os dados relacionais quanto os vetores de alta dimensionalidade (*embeddings*) das produções científicas.
- **FastAPI (Python)**: API RESTful que gerencia as regras de negócio, expõe os endpoints de busca textual (POST `/api/search/text`) e busca semântica (POST `/api/search/semantic`), e serve a camada de front-end.
- **Worker de Embeddings**: script Python acionado após cada carga ETL que identifica produções sem vetores e gera seus *embeddings* via **Sentence-Transformers** (modelo local), com *fallback* automático para **OpenAI / Langchain** em caso de falha.
- **Gerador CSV**: processo interno que consolida os dados em uma base multidimensional e exporta arquivos CSV com as dimensões Pesquisador e Quadrienal-Ano e o fato Produção Científica, para consumo no Power BI.

5.3 Camada Front-End

Responsável pela interação e visualização dos dados. Subdivide-se em:

- **Next.js (Vercel):** aplicação web moderna com renderização do lado do servidor que consome a API REST da camada back-end via JSON e exibe os resultados de busca em cards responsivos com indicadores Qualis e JCR.
- **Power BI:** ferramenta de análise que consome os arquivos CSV gerados pelo back-end para elaboração de painéis interativos com gráficos de publicações por ano, por pesquisador e distribuição por estrato Qualis.

5.4 Protocolos de Comunicação

Tabela 4: Protocolos de comunicação entre as camadas

Origem	Destino	Protocolo / Formato
Apache Hop	Supabase	Conexão direta PostgreSQL (JDBC)
Apache Hop	CrossRef/OpenAlex	HTTPS / REST JSON
FastAPI	Supabase	Conexão direta PostgreSQL (asyncpg)
FastAPI	Sentence-Transformers	Chamada local (Python)
FastAPI	OpenAI	HTTPS / REST JSON
Next.js	FastAPI	HTTPS / REST JSON
Power BI	Gerador CSV	Arquivo CSV (download)

6 Esquema do Banco de Dados

O banco de dados do sistema SPARK é hospedado no **Supabase** (PostgreSQL gerenciado) com a extensão **pgvector** habilitada. O modelo é composto por 7 tabelas, sendo **auth.users** gerenciada internamente pelo Supabase Auth e as demais criadas pelo script DDL do projeto.

6.1 Diagrama Entidade-Relacionamento

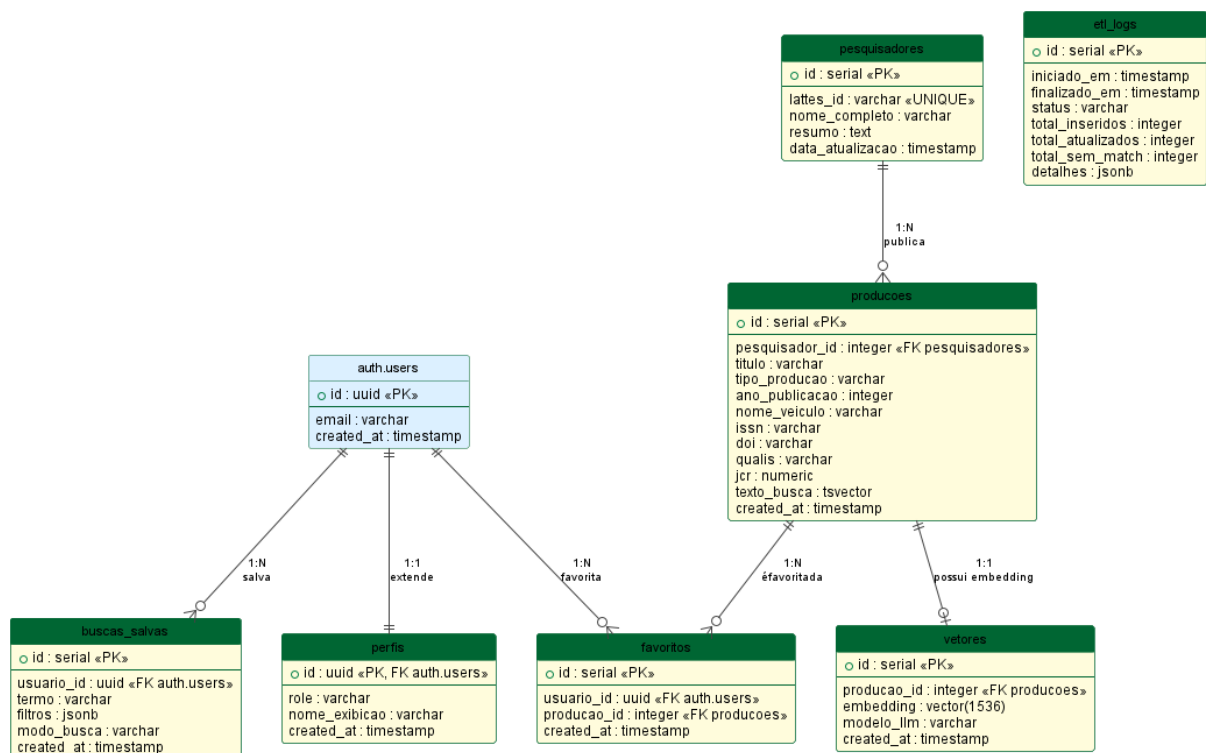


Figura 3: Diagrama Entidade-Relacionamento do sistema SPARK

6.2 Descrição das Tabelas

Tabela 5: Tabelas do banco de dados SPARK

Tabela	Descrição
<code>auth.users</code>	Gerenciada pelo Supabase Auth. Armazena e-mail, senha (hash) e metadados de autenticação dos usuários.
<code>perfis</code>	Estende o <code>auth.users</code> com o papel do usuário no sistema (<code>usuario</code> ou <code>admin</code>) e nome de exibição.
<code>pesquisadores</code>	Dados dos pesquisadores extraídos do currículo Lattes: nome completo, identificador Lattes e resumo do currículo.
<code>producoes</code>	Produções científicas dos pesquisadores com título, tipo, ano, veículo, ISSN, DOI, estrato Qualis, Fator de Impacto JCR e coluna <code>tsvector</code> para Full-Text Search.
<code>vetores</code>	Embeddings vetoriais das produções gerados pelo modelo LLM, armazenados como <code>vector(1536)</code> via extensão <code>pgvector</code> para busca semântica por similaridade de cosseno.
<code>favoritos</code>	Relacionamento entre usuários autenticados e produções favoritas. Possui restrição de unicidade por par (usuário, produção).
<code>buscas_salvas</code>	Buscas salvas pelos usuários autenticados, armazenando o termo, os filtros aplicados em formato JSONB e o modo de busca (textual ou semântico).
<code>etl_logs</code>	Histórico de execuções do pipeline ETL com status, totais de registros inseridos, atualizados e sem correspondência, e detalhes em formato JSONB.

6.3 Decisões Técnicas do Modelo

- **tsvector com trigger:** a coluna `texto_busca` na tabela `producoes` é populada automaticamente por um trigger a cada inserção ou atualização, garantindo que o índice GIN de Full-Text Search esteja sempre atualizado sem intervenção manual.

- **IVFFlat para busca vetorial:** o índice `ivfflat` com `vector_cosine_ops` na tabela `vetores` permite buscas por similaridade de cosseno com alta performance, mesmo com grande volume de produções.
- **Row Level Security (RLS):** todas as tabelas possuem RLS habilitado. Produções e pesquisadores têm leitura pública. Favoritos e buscas salvas são visíveis apenas pelo próprio usuário. Logs do ETL são restritos ao papel `admin`.
- **JSONB para flexibilidade:** os campos `filtros` (buscas_salvas) e `detalhes` (etl_logs) usam JSONB para armazenar dados semi-estruturados sem necessidade de migrações de schema a cada evolução do sistema.

7 Rastreabilidade dos Requisitos

A tabela a seguir relaciona cada História do Usuário com o Sprint em que será implementada, conforme o planejamento do projeto.

Tabela 6: Rastreabilidade entre Histórias do Usuário e Sprints

ID	Sprint	Descrição	Prioridade
US-01	Sprint III	Busca textual (Full-Text Search)	Alta
US-02	Sprint III	Busca semântica via pgvector	Alta
US-03	Sprint II	ETL dos dados Lattes via Apache Hop	Alta
US-04	Sprint V	Geração de CSVs e painel Power BI	Média

8 Considerações Finais

Este documento constitui a base de especificação do Sprint I do projeto SPARK e deverá ser atualizado a cada Sprint Review conforme o professor valide ou solicite ajustes nos requisitos. Todos os artefatos produzidos serão versionados no repositório Git da equipe, com commits referenciando as respectivas issues do Jira (projeto SPARK).